Project Apollo

Integration Strategy

Deloitte Deliverable E6

ServiceStream

Document Control Information

Project	Project Apollo
Stream	PMO
Deliverable ID	E6
Jira ID	ETF-111
Deliverable Title	Integration Strategy
Deliverable Description	The Integration Strategy document provide general design approaches that will be taken for the various integration components of the application including for Web services, messaging, service orchestration, and batch. This document contains detailed technical information that will help the technical team (including integration architects and developers) understand the integration architecture and design and to implement it properly
Document Version	V 1.0
Document Status	Ready for Approval
SharePoint Link	Project Apollo - E6 - Integration Strategy v1.0.docx

Edit History

Version	Date	Additional/Modifications	Prepared/Revised By
0.1	08/04/2026	Initial Draft	Will Soehendra
1.0	17/04/2026	Feedback from Ankit S and Rajesh S incorporated, for formal submission	Will Soehendra
1.1	4/05/2026	Feedback from SSM review incorporated.	Will Soehendra

Document Review

	Name	Title/Role
Reviewed By		
Signature		

Document Sign Off

	Name	Title/Role
Approved By		
Signature		**Date:**

Table of Contents

1. Overview

1.1. Purpose/Objective

The purpose of this document is to provide strategy and approach for the proposed future state Integration for the Project Apollo landscape.

This document serves as an overview document for use by the technical team to understand current and future integration scenarios and aims to assists developers to understand various technical integration solutions available for interfaces. Additionally, the maintainability and quality of development can be enhanced by adhering to a common set of standards.

1.2. Audience

This document is written for business IT and System Integrator architects, leads and developers directly involved with the design, implementation, and on-going operations of the integration landscape within Service Stream.

1.3. Scope

The scope of this document comprises of the Project Apollo system scope, and all integrations within these systens.

In the Project Apollo system scope, this specifically includes:

System / Approach	Integration Purpose
1. Non-FSM Integration	Integrations to operational systems (non-FSM)
Beakon	To replicate onboarded suppliers from Beakon to S/4HANA, and sync supplier statuses.
BlackLine	To sync financial (G/L) data, for financial reconciliation in Blackline.
SecurePay/Fat Zebra* ** Pending ETF-442*	To authorize payments for Digital Stores purchases
SuccessFactors EC/ECP	To replicate employees master records to S/4HANA and integrating S/4 master data to SuccessFactors for Employee master data.
ACMO	To post approved Supplier Invoices into S/4HANA and integrating S/4 purchasing/financial data to ACMO for three-way matching.
SAP Workforce Software	To integrate Service Order/Project WBS to Workforce Software and time confirmations to S/4HANA based on employee timesheets.
SAP Concur	To integrate employee expense postings to S/4HANA and replicate master data to Concur.
Bank Account Integration (via Multi Bank Connectivity (16R)	To integrate bank statements and payment files with ANZ.
SAP Business Technology Platform	To integration S/4HANA with BTP apps (such as Digital Stores) and the Work Zone landing page.
SAP Identity Access Management	To integrate user authentication centrally with Service Stream corporate IDP.

2. D4C Replication	To replicate data objects from S/4HANA to D4C's Data Warehouse, replacing the Jobpac dataset.
3. FSM Integration	Integration specifically to FSM systems, for service management to be carried out and reflected in S/4HANA.
JAMS transformation (~~file based~~ API Based)	To replicate work orders, purchase orders, work confirmation and customer billing to S/4HANA.
Service Now transformation (API based)	To replicate work orders, purchase orders, work confirmation and customer billing to S/4HANA.
SiteTracker (API based)	To replicate work orders, purchase orders, work confirmation and customer billing to S/4HANA.
Canonical Models	To standardise FSM to S/4HANA integration.
MDFS	To integrate work order lifecycle, subcontractor claims, work confirmation and billing processes from MDFS into S/4HANA. Integration is triggered at key business events (such as scheduling, work completion and SCC approved status), with bulk processing aligned to current Jobpac practices. MDFS will provide data via file-based extraction, which will be processed through SAP Cloud Integration using the canonical model approach. These canonical interfaces will standardise processing into S/4HANA for service order creation and update, procurement (PO/SES), work confirmation and billing. Supplier invoice and payment references will be written back to MDFS via API to support claim closure.
PIMS	To integrate subcontractor claim processing and payment lifecycle from PIMS into S/4HANA. Integration is triggered at the payment-ready stage, aligned to current operational practices. PIMS will provide data via database extract, which will be processed through SAP Cloud Integration using the canonical model approach. These canonical interfaces will standardise processing into S/4HANA for procurement (PO/SES), supplier invoicing (including RCTI) and financial postings. Payment status and financial references will be written back to PIMS via file-based or API mechanism to update claim status and support closure. Customer revenue will continue to be processed via structured file upload into S/4HANA, consistent with the current Jobpac approach.

Commented [WS1]: @Rajesh Sha Can you please fill in integration scope for MDFS/PIMS?

Commented [RS2R1]: @William Soehendra Updated

2. Integration Scenarios

2.1. Integration Pattern Matrix

The following integraiton scenarios have been defined for Project Apollo, to categorise individual interfaces into general types of S/4HANA data flow and method.

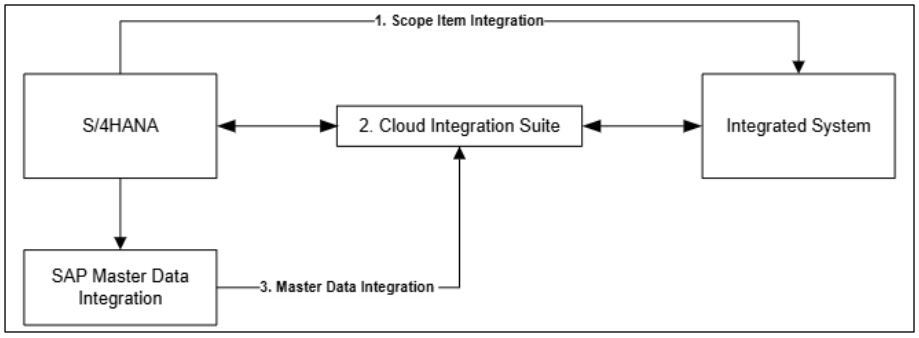
During the Explore phase, and as part of the Extension Inventory deliverable, interfaces will be categorised into one of the following patterns.

Pattern ID	Scenario	Details
STANDARD	Standard	Where standard native integration is configured between SAP to SAP systems, via: Cloud Integration, Master Data Integration (MDI) or direct Scope Item integration.
IN-API	Inbound API	Where S/4HANA receives data via an API, triggered by a Cloud Integration interface. This includes “Canonical Model” integrations into S/4HANA, via API.
OUT-API	Outbound API	Where a Third-Party system receives S/4HANA data exposed via an API, triggered by a Cloud Integration Interface.
FSM-E2E	Inbound File	Custom built, reusable integration pattern for FSM integrations into S/4HANA, from Cloud Integration. This includes data pickup interfaces from source FSM to Cloud Integration, as well as Canonical interfaces from Cloud Integration to S/4HANA.
BDC	Data Replication via BDC	Where data replication (of CDS views from S/4HANA) occurs through Business Data Cloud to an external system/database. This includes both Service Stream data replication, and D4C data replication.
BOOMI	Boomi-Driven	Where integration is primarily driven by the existing Boomi middleware, interacting with S/4HANA or Cloud Integration.

2.2. Pattern STANDARD

SAP provides native integrations in the form of pre-packaged interfaces, managed integration services on BTP, or direct integrations to S/4HANA with Scope Item integration.

These are integrations between systems that do not require custom build on Cloud Integration or any other middleware tool. Only configuration within S/4HANA, other SAP systems and BTP is required for setup.



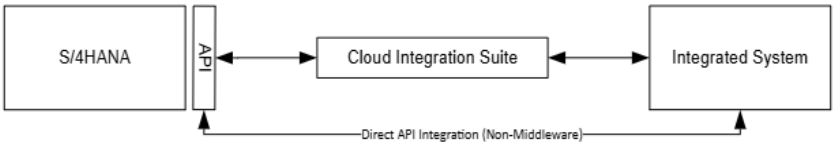
Integration of type STANDARD can follow one of three integration patterns:

- **Scope Item Integration**: Utilising S/4HANA Scope Items and embedded connectivity to integrate with external systems.
- **Cloud Integration Suite**: Utilising a pre-packaged, configurable integration flow on Cloud Integration Suite (from SAP) to integrate with external systems.
- **Master Data Integration**: Utilising SAP Master Data Integration (on BTP) to sync and distribute S/4HANA master data to external systems. Once master data has been replicated to MDI, systems can consume directly, or consume via a Cloud Integration interface.

2.3. Pattern IN-API and OUT-API

S/4HANA provides standard APIs that can be consumed to manipulate data and trigger processes. These APIs are accessible publicly and can utilise Basic/OAuth token authentication. Where no standard integration exists, integrations to and from S/4HANA will utilise these APIs, called from Cloud Integration in a custom interface, or from third party systems directly.

APIs on S/4HANA are, by default, not enabled (or exposed). Configuration of Communication Arrangements, Communication Systems and Communication Users is required to enable these APIs – this is performed on the S/4HANA system.



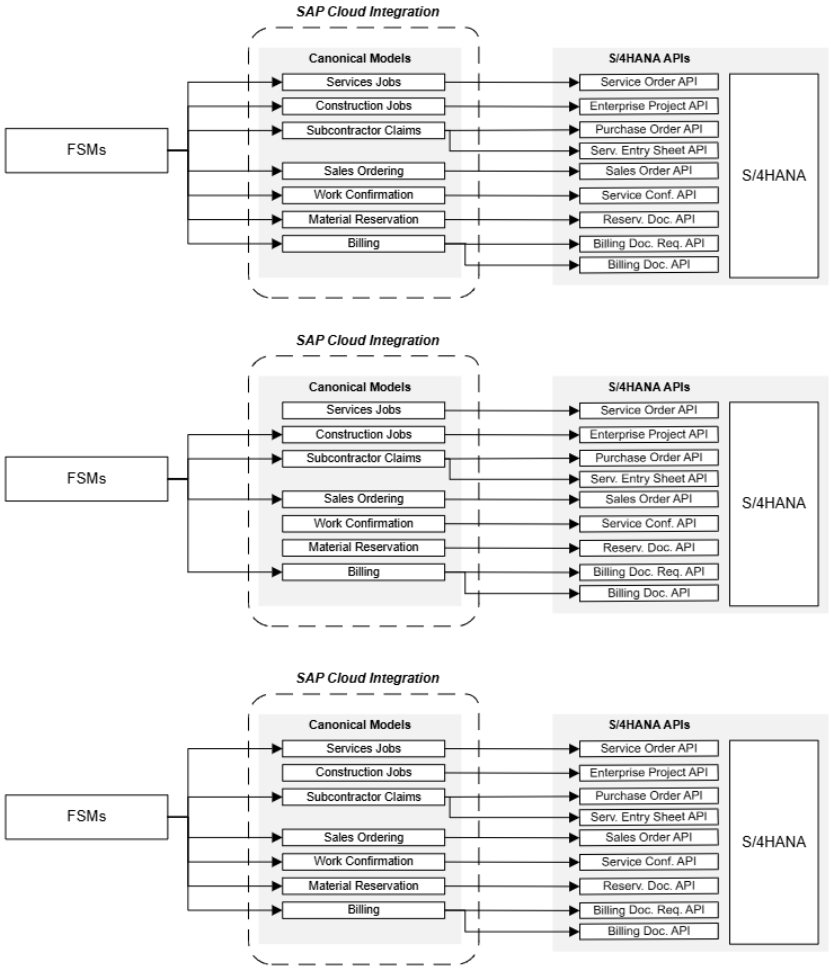
2.4. FSM-E2E

The FSM-E2E integration pattern has been defined to cater specifically for the Field Service Management integration to S/4HANA, via the Canonical Model approach. This pattern and approach promotes reusability, flexibility and simplicity for the high number of integration points related to FSMs.

This integration pattern includes both:

- Integration from FSM to Cloud Integration: Utilising Cloud Integration receive messages from FSMs, or to query at a predefined interval/trigger event.
- Integration from Cloud Integration to S/4HANA (Canonical): Utilising Cloud Integration to provide a predefined interface model that triggers one or more S/4HANA APIs.

By design, each FSM Interacting with a canonical model can use all, or some, of the models to interact with S/4HANA.

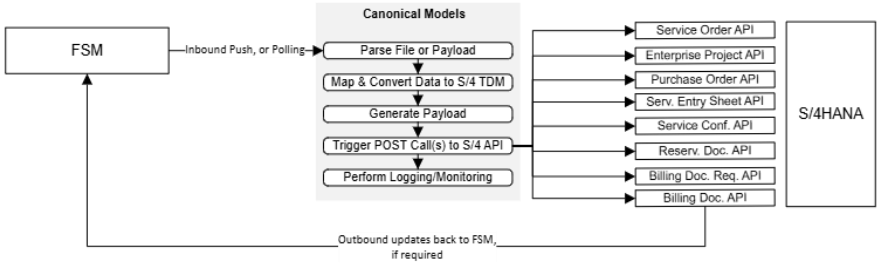


2.4.1. Canonical Model Approach for FSM to/from S/4HANA Integrations

Canonical Models is the approach that has been defined for FSM to S/4HANA Cloud integrations, emphasising:

- Reusable integration artefacts
- Standardisation of integration across multiple FSMs
- Flexiblity and abstraction of FSM inputs into S/4HANA

For Project Apollo, this canonical model approach is to be used primarily for JAMS, ServiceNow and Sitetracker, as well as all other FSMs currently in-place and to be integrated with S/4HANA. However, in future, the same canonical models that have been designed for Project Apollo may be used for new FSMs integrating with S/4HANA Cloud.



A canonical model aims to provide a standard interface point into S/4HANA Cloud, by accepting a pre-defined input and interacting with pre-defined S/4HANA APIs. This means that, as long as an FSM can send expected data into the model, a defined process in S/4HANA Cloud can be triggerred to reflect an event or process.

Canonical models will exist as integration flows on Cloud Integration – exposed as an endpoint, which then processes incoming data and posts against an S/4HANA Cloud API. This integration flow consists of data mapping, data transformation and payload generation, to fit the S/4HANA Cloud API payload requirements.

Each canonical model is to be trigerred by a preceeding interface, which interacts with the FSM system. This preceeding flow, existing as a separate integration flow, is designed to query/retrieve data from FSMs for the canonical model, or process incoming messages before trigerring the canonical model.

Six canonical models have been identified for Project Apollo, in-line with the functional process design for service management. These models are:

ID	Interface	Source System	Target System	Category
INT-21	Canonical Service Order Creation/Update	Cloud Integration	S/4HANA	FSM E2E
INT-22	Canonical Enterprise Project Creation/Update	Cloud Integration	S/4HANA	FSM E2E
INT-23	Canonical Purchase Order and SES Creation	Cloud Integration	S/4HANA	FSM E2E
INT-24	Canonical Work Confirmation - Service and Projects	Cloud Integration	S/4HANA	FSM E2E
INT-25	Canonical Material Reservation	Cloud Integration	S/4HANA	FSM E2E
INT-26	Canonical Work Billing - Service and Projects	Cloud Integration	S/4HANA	FSM E2E
INT-65	Canonical Sales Order	Cloud Integration	S/4HANA	FSM E2E

Each canonical model interacts with defined S/4HANA Cloud APIs:

ID	Interface	S/4 API	Operation
INT-21	Canonical Service Order Creation/Update	Service Order (A2X) - Service Order Service Order (A2X) - Service Order Items	CREATE UPDATE
INT-22	Canonical Enterprise Project Creation/Update	Enterprise Project - Project Enterprise Project - Project Element	CREATE UPDATE
INT-23	Canonical Purchase Order and SES Creation	Purchase Order - Purchase Order Purchase Order - Purchase Order Items Service Entry Sheet (Lean Services) - Service Entry Sheet Service Entry Sheet (Lean Services) - Items	CREATE
INT-24	Canonical Work Confirmation - Service and Projects	Service Order (A2X) - Service Confirmation	CREATE
INT-25	Canonical Material Reservation	Reservation Document - Header Reservation Document - Items	CREATE
INT-26	Canonical Work Billing - Service and Projects	Billing Document Request Billing Document	CREATE
INT-65	Canonical Sales Ordering	Sales Order	CREATE

At time of writing, the functional detail of the canonical models are still to be finalised – therefore, the final list of models, and the S/4 APIs to be triggerred, may change. Subsequent updates will be recorded in this document in future.

Design details for each canonical model, including payload structures, will be documented in the corresponding Functional Specification Document.

2.5. BDC

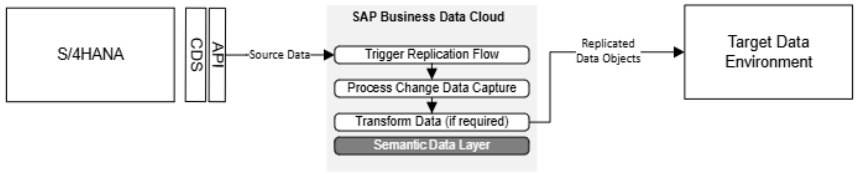
BDC data replication is the approach defined to replicate data obejcts from S/4HANA to Service Stream and external target data environments. With this pattern, data objects are exposed from S/4HANA using standard SAP mechanisms (such as CDS-based data products), and BDC manages the initial load and subsequent deltas using change data capture rather than repeated full extracts.

This allows BDC to reliably keep replicated objects in sync while minimizing load on the source system and avoiding custom point-to-point integrations. Within BDC, these replication flows are centrally configured and monitored.

In BDC, data can be transformed, harmonized, and aligned to common business semantics before being delivered to the target environment. After replication, the prepared data objects can be replicated onward to the target system—such as a cloud data warehouse, lakehouse, or analytics platform—using controlled, repeatable replication patterns.

In this way, BDC functions as middleware, decoupling S/4HANA from downstream data systems, absorbs changes in source structures, and provides a stable, governed replication interface for multiple targets without tightly coupling them to the transactional core.

Project Apollo will use BDC to replicate data objects from S/4HANA to Service Stream's Analytics Data Environment, and to D4C's Data Warehouse for downstream consumption.



2.5.1. Datasphere

SAP Datasphere provides a governed, business-ready data layer that can connect, harmonise and share data across SAP and non-SAP platforms, making it a practical option for supporting data movement between SAP S/4HANA (and other SAP source systems) and a Data Lake. For Project Apollo, Datasphere will be used to replicate selected S/4HANA data into downstream lake environments within Service Stream (AWS Redshift and Microsoft Fabric) and D4C (Azure Data Lake) primarily.

This approach supports near real-time or scheduled replication, reduces manual integration effort, and provides a more controlled foundation for extending enterprise data beyond core ERP systems.

2.6. ~~BOOMI~~bound

Boomi is the existing middleware platform currently in place within the Service Stream landscape, supporting integrations across legacy systems including Jobpac, BlackLine and ACMO.

For Project Apollo, SAP Cloud Integration will be the primary integration platform for S/4HANA. Boomi will continue to be utilised where existing integrations are already established and provide a simpler or more efficient transition approach.

This includes:

BlackLine
Boomi will continue to support existing integrations for financial data movement from legacy systems into BlackLine during the transition. Where required, Boomi may be used to route or transform data before interfacing with S/4HANA or SAP Cloud Integration. The target state will align to S/4HANA as the single source of truth for financial data feeding BlackLine.

ACMO
Boomi will continue to support supplier invoice processing integrations between ACMO and S/4HANA. This includes routing of approved invoices into S/4HANA and synchronisation of

procurement and financial data required for three-way matching. Existing Boomi integrations will be leveraged to minimise disruption during transition.

Boomi will operate alongside SAP Cloud Integration in a complementary role, supporting coexistence with legacy systems while the target-state integration model is established.

The strategic direction remains:

- SAP Cloud Integration as the standard integration platform for S/4HANA
- Boomi retained for existing integrations and transition scenarios only

3. Interface Listing

As part of F2S and the Explore design phase, confirmed interfaces have been validated.

Note: the final list of interfaces can change, as will be updated in the Extension Register.

ID	Interface	Source System	Target System	Category
INT-01	GL Balances	S/4HANA	Blackline	OUT-API
INT-02	GL (Subledger) Open Items	S/4HANA	Blackline	OUT-API
INT-03	G/L Account Master	S/4HANA	Blackline	OUT-API
INT-04	Cost Center Replication	S/4HANA	SuccessFactors EC	Standard
INT-05	G/L Master Data	S/4HANA	SuccessFactors EC	Standard
INT-06	Bank Master Data	S/4HANA	SuccessFactors EC	Standard
INT-07	Employee Replication	SuccessFactors EC	S/4HANA	Standard
INT-08	Org Structure/Assignments Replication	SuccessFactors EC	S/4HANA	Standard
INT-09	Cost Center Replication	S/4HANA	SuccessFactors EC Payroll	Standard
INT-10	Payroll Postings	SuccessFactors EC Payroll	S/4HANA	Standard
INT-11	Bank Connectivity: ANZ	S/4HANA	MBC	Standard
INT-12	[TBD] Payment Processing	SecurePay/Fat Zebra	S/4HANA	IN-API
INT-13	SiteTracker (ODM) Project and Job Creation/Update Processing	SiteTracker	Cloud Integration	FSM E2E
INT-14	SiteTracker (ODM) Artefact Submission Processing	SiteTracker	Cloud Integration	FSM E2E
INT-15	SiteTracker (ODM) Material Consumption Processing	SiteTracker	Cloud Integration	FSM E2E
INT-16	SiteTracker (ODM) Work Completion Processing	SiteTracker	Cloud Integration	FSM E2E
INT-17	SiteTracker (Wireless) Project and Job Creation/Update Processing	SiteTracker	Cloud Integration	FSM E2E
INT-18	SiteTracker (Wireless) Artefact Submission Processing	SiteTracker	Cloud Integration	FSM E2E
INT-19	SiteTracker (Wireless) Material Consumption Processing	SiteTracker	Cloud Integration	FSM E2E
INT-20	SiteTracker (Wireless) Work Completion Processing	SiteTracker	Cloud Integration	FSM E2E
INT-21	Canonical Service Order Creation/Update	Cloud Integration	S/4HANA	FSM E2E
INT-22	Canonical Enterprise Project Creation/Update	Cloud Integration	S/4HANA	FSM E2E
INT-23	Canonical Purchase Order and SES Creation	Cloud Integration	S/4HANA	FSM E2E

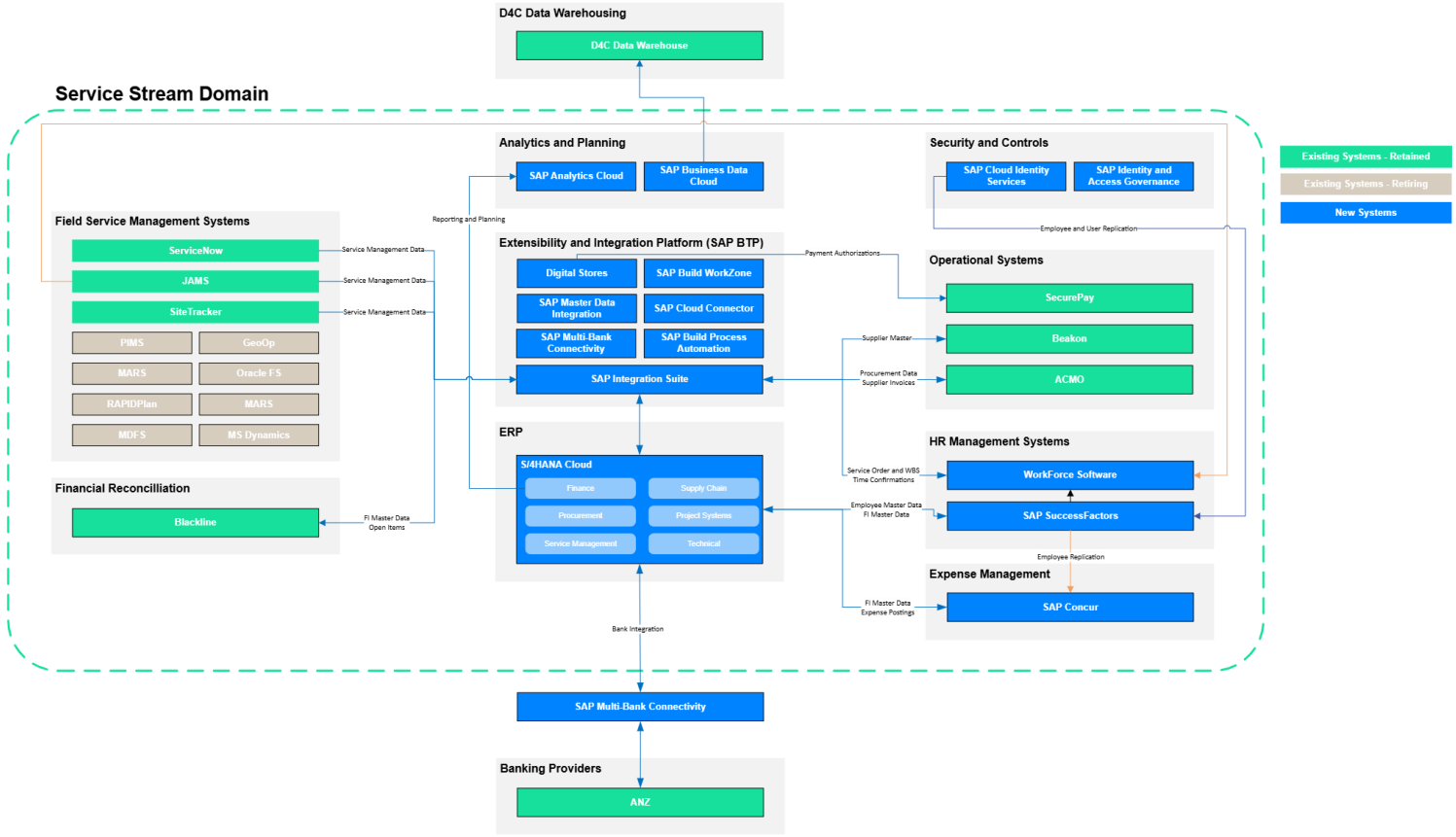
INT-24	Canonical Work Confirmation - Service and Projects	Cloud Integration	S/4HANA	FSM E2E
INT-25	Canonical Material Reservation	Cloud Integration	S/4HANA	FSM E2E
INT-26	Canonical Work Billing - Service and Projects	Cloud Integration	S/4HANA	FSM E2E
INT-27	Servicenow (ITSM) Work Creation Processing	Cloud Integration	S/4HANA	FSM E2E
INT-28	Servicenow (ITSM) AP Lines Processing	Cloud Integration	S/4HANA	FSM E2E
INT-29	Servicenow (ITSM) Claim Billing Processing	Cloud Integration	S/4HANA	FSM E2E
INT-30	JAMS Job Creation/Updates (Services) Processing	JAMS	Cloud Integration	FSM E2E
INT-31	JAMS Subcontractor Claim Processing	JAMS	Cloud Integration	FSM E2E
INT-32	JAMS Material Consumption Processing	JAMS	Cloud Integration	FSM E2E
INT-33	JAMS Billing Processing	JAMS	Cloud Integration	FSM E2E
INT-34	JAMS Job Creation (Project) Processing	JAMS	Cloud Integration	FSM E2E
INT-35	Cost Object Export to Concur	S/4HANA	Concur	STANDARD
INT-36	Expense Report Postings	Concur	S/4HANA	STANDARD
INT-37	Financial Configuration Export	S/4HANA	Concur	STANDARD
INT-38	Company Code Replication	S/4HANA	ACMO	OUT-API
INT-39	GL Account Replication	S/4HANA	ACMO	OUT-API
INT-40	Tax Code Replication	S/4HANA	ACMO	OUT-API
INT-41	Payment Terms Replication	S/4HANA	ACMO	OUT-API
INT-42	Supplier Master Replication	S/4HANA	ACMO	OUT-API
INT-43	Purchase Order Replication	S/4HANA	ACMO	OUT-API
INT-44	Goods Receipts Replication	S/4HANA	ACMO	OUT-API
INT-45	Supplier Invoice Status Replication	S/4HANA	ACMO	OUT-API
INT-46	Supplier Invoice Posting	ACMO	S/4HANA	IN-API
INT-47	Supplier Invoice Attachment Posting	ACMO	S/4HANA	IN-API
INT-48	Supplier Creation/Update	Beakon	S/4HANA	IN-API
INT-49	Supplier Updates	S/4HANA	Beakon	OUT-API
INT-50	Project Code Replication for Timesheets	S/4HANA	WFS	OUT-API
INT-51	Service Order Replication for Timesheets	S/4HANA	WFS	OUT-API
INT-52	Labour Timesheets	WFS	S/4HANA	IN-API
INT-53	Equipment Usage	WFS	S/4HANA	IN-API
INT-54	D4C Data Replication*	S/4HANA	D4C Data Warehouse	BDC
INT-55	Service Stream Data Replication*	S/4HANA	SSM Data Lake	BDC
INT-56	MDFS Service Order Creation/Update Processing	MDFS	Cloud Integration	FSM E2E
INT-57	MDFS Work Confirmation Processing	MDFS	Cloud Integration	FSM E2E
INT-58	MDFS Subcontractor Claim Processing	MDFS	Cloud Integration	FSM E2E
INT-59	MDFS Work Billing Processing	MDFS	Cloud Integration	FSM E2E
INT-60	MDFS Customer Billing Upload	MDFS	S/4HANA	Manual
INT-61	MDFS Supplier Invoice Write-back	S/4HANA	MDFS	OUT-API
INT-62	PIMS Subcontractor Claim Processing	PIMS	Cloud Integration	FSM E2E
INT-63	PIMS Payment Status / Reference Update	S/4HANA	PIMS	OUT-API
INT-64	PIMS Customer Revenue Upload	External	S/4HANA	Manual

Commented [WS3]: @Rajesh Sha Can you please add the MDFS/PIMS interfaces here?

Commented [RS4R3]: @William Soehendra Sent you list of interfaces for PIMS and MDFS, include them here please

4. Integration Landscape

To-Be Architecture Diagram



5. Integration Services, Platforms and Tools

5.1. S/4HANA Cloud Integration Framework

S/4HANA Public Cloud operates as a standardised SaaS environment where all integrations occur through public APIs that are pre-built and maintained by SAP. S/4HANA Cloud enforces a strict “API-first“ approach to integration within the system.

Standard APIs expose business functionality as consumable services that external systems can call. Full documentaion and detail on the published APIs are available on the SAP Business Accelerator Hub.

5.1.1. S/4HANA Cloud API Layer

API connectivity within S/4HANA is controlled by the Communication Arrangement framework, which are the mechanisms for activating and configuring API access. Each Communication Arrangement specifies which API is enabled and configured for consumption by external systems. SAP maintains the Communication Arrangements, which are bundled into the Scope Items activated during configuration in CBC.

To establish an integration, connectivity administrators in S/4HANA Cloud create a Communication System (representing the external application), a Communication User (credentials), and then bind these together in a Communication Arrangement. This configuration is performed within S/4HANA Cloud's Fiori launchpad under “Communication Management.“

5.2. Integration Tools

The following integration tools/services are provided by SAP and will be leveraged by Project Apollo:

5.2.1. SAP Cloud Integration Suite

SAP Cloud Integration (CI) Suite is a BTP service that provide a middleware platform to design, develop, deploy and monitor integrations across the Project Apollo landscape. This is the SAP standard platform for real-time integration.

Project Apollo will utilise SAP CI for all custom-built integrations into and out of S/4HANA.

5.2.2. SAP Master Data Integration

SAP Master Data Integration (MDI) is a BTP service to act as a central hub for the distribution of common master data across system landscapes. With pre-defined object types, MDI connects directly with master data sources (such as S/4HANA and SuccessFactors) and can be connected to by downstream systems for consumption (including Cloud Integration). With the pre-defined object types, MDI manages changes from source systems to be distributed, with logging and monitoring available.

Project Apollo will utilise SAP MDI to distribute SAP master data (such as Employees from SuccessFactors and Cost Centers from S/4HANA) to downstream systems.

5.2.3. S/4HANA Cloud APIs

The S/4HANA Cloud solution comes with pre-built standard APIs that allows data to be read, and transactions to be triggered from external systems and integrations. These APIs are delivered by SAP, and are continually maintained as the product evolves with subsequent upgrades.

Project Apollo will utilise these S/4HANA standard APIs for the scope of integrations with S/4HANA.

5.2.4. SAP Business Data Cloud

BDC provides standardized services such as replication flows, change data capture, and push-based replication mechanisms from SAP source systems—that move data objects from S/4HANA to target

data environments. This is a simpler approach to integrations where data replication is required from S/4HANA, and is aligned to SAP best practices architectural patterns.

Project Apollo will utilise BDC as the middleware/data replication tool for Service Stream target data environments, as well as the D4C target data environment.

5.2.5. Dell Boomi

Dell Boomi is the existing middleware platform currently in place at Service Stream, supporting integrations across legacy systems including Jobpac, BlackLine and ACMO.

Whilst the target architecture for Project Apollo is centred on SAP Cloud Integration as the primary integration platform for S/4HANA Cloud, there are specific scenarios where leveraging the existing Boomi integrations provides a simpler and more efficient transition approach.

In these instances, existing Boomi integrations will be retained and used to:

• Interact directly with S/4HANA via standard APIs, where appropriate
• Interface with SAP Cloud Integration to trigger canonical model-based processing
• Support continuity of existing integration patterns without requiring unnecessary redesign during transition

For Project Apollo, Boomi will continue to support:

• **BlackLine** – financial data integration and reconciliation processes, transitioning towards S/4HANA as the primary source of financial data
• **ACMO** – routing and processing of supplier invoices, including integration with ~~S/4HANA~~CPI for posting and status synchronisation

Boomi will operate alongside SAP Cloud Integration in a complementary role, supporting coexistence with legacy systems while the target-state integration model is established.

The strategic direction remains:

• SAP Cloud Integration as the standard integration platform for all new S/4HANA integrations
• Canonical model approach for FSM integrations via SAP Cloud Integration
• Boomi retained for existing integrations and transition scenarios only, with no expansion of its footprint in the target architecture

Commented [RS5]: @William Soehendra Reviewed and updated

5.3. Pre-packaged Content for Integration

SAP Cloud Integration includes a number of pre-packaged integration flows that are designed to connect S/4HANA to other SAP and non-SAP solutions. These are maintained by SAP, and require only configuration to setup and deploy. However, these prepackged integrations are not able to be modified.

Project Apollo has a number of in-scope integrations (such as S/4HANA and Concur) that exist as prepackaged integration flows on CI. These will be set up and configured for use.

5.4. Decision Matrix for Choosing ETL Tool

Integration Methodology	Recommendations of Integration
Standard Pre-package Integration content	• Opting for SAP Cloud Integration is recommended when pre-packaged content (i.e., integration flow templates) is available, as it reduces both development time and cost. • SAP regularly releases updated integration content to ensure continued improvements. • SAP provides ongoing support for its integration content.
Use Native Adapters to Connect with SAP	• When there is a SAP related system, it is advisable to use the native/proprietary adapters provided by SAP, such as IDOC, RFC, or SuccessFactors. • Leveraging SAP CPI ensures timely support and seamless native connectivity with SAP systems.
Custom Development	• When no integration content is available from SAP, custom development is required. • SAP CPI is recommended when no existing integration is in place. • If an existing integration exists through another medium, it should be maintained.

6. Interface Development Guidelines and Naming Standards

6.1. SAP Cloud Integration

6.1.1. Packages

Packages are a collection for integration content on Cloud Integration – holding integraiton flows, value mappings, oData services, and any other artefact. Packages will be used to hold logical groupings of distinct integrations – for example, one package to hold all canonical model integration flows, value maps, and any other relevant content.

6.1.2. Package Naming Conventions

For detailed naming convention at Service Stream / Project Apollo S/4 Implementation for all SAP Cloud Platform Integration, API Management and Event Mesh development objects, all developers willl to follow the following format:

	Convention	Example
Non-Productive	" <System_Name1> to <System_Name2> - <Environment> ~~<System_Name1> Integration with <System_Name2> - <Environment>~~"	*Concur Integrations with S4HANA Cloud – Dev.*
Productive	<System_Name1> to <System_Name2> ~~"<System_Name1> Integrations with <System_Name2>"~~	*Concur Integrations with S4HANA Cloud*

6.1.3. Package Short Description Guidelines

This is the text that is displayed on the package tile in the hub and is particularly important because it is the first description that support team may use to identify and discover existing interfaces to promote reusability.

6.1.4. Integration Flow Naming Conventions

	Convention	Example
Non-Productive	<INT-xx> - <Sender system> - <Receiver system> - <Data Object> - Environment	*INT-01 – S4Hana – Blackline – GLBalances - Dev*
Productive	<INT-xx> - <Sender system> - <Receiver system> - <Data Object>	*INT-01 – S4Hana – Blackline – GLBalances*

~~6.1.4.~~6.1.5. General Step Description Guidelines

The step names inside the Integration flow and descriptions in the Integration Flow to provide meaningful context. It should follow the below guidelines in addition to the English grammar rules:

- Start description sentences with a capital, end with a period.

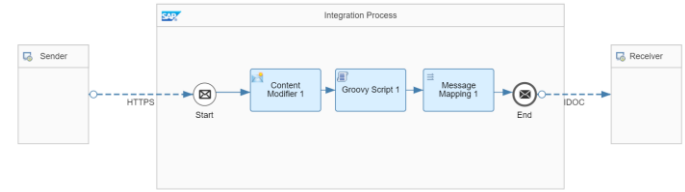
- Avoid describing low level implementation details and dependencies unless they are important for usage.
- Avoid repetitions and misplacements of information: for example, do not write about parameters in an operation description
- Describe the objective of a step or the task that is executed by a step in the integration flow in plain English.

~~6.1.5.~~6.1.6. Integration Flow Look and Feel Guidelines

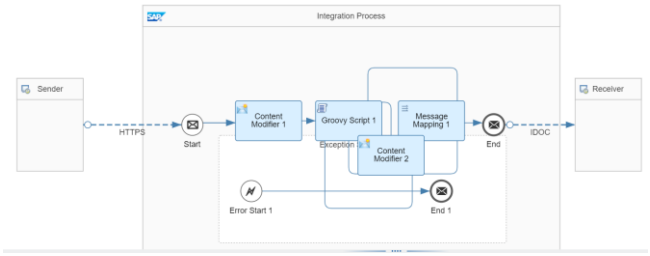
The following guidelines should be used to design integration flow layout for simplifying maintenance.

- integration flows logically grouped under a Package
- Create a POC or test package for duplicating or creation of POC INTEGRATION FLOWs.
- No editing of standard, pre-packaged integration flows form SAP – only configuration. Edited flows cannot have updates applied
- Left to right flow direction for readability, with the sender system to the extreme left and target system on extreme right.
- Integration flow Sender address url formatting to follow:
  " <environment>/<version>/~~<environment>/<system_name>/~~<integration flow_name> "

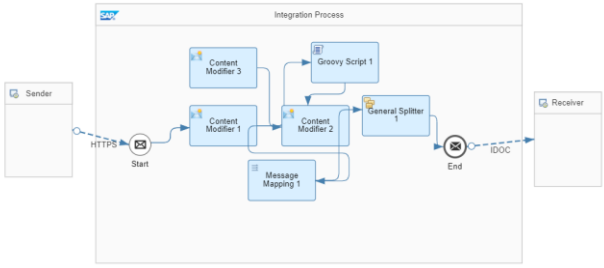
  E.g. /v1/dev/concur/EmployeeMasterdata



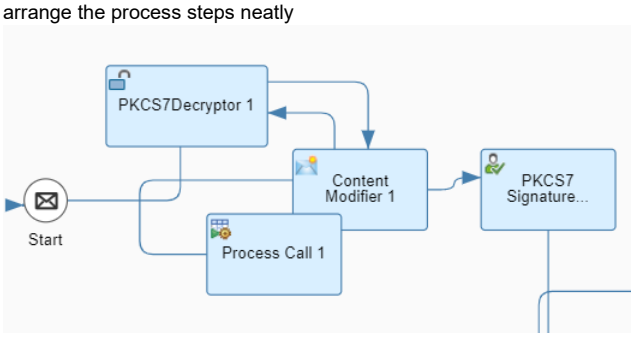
- Avoid overlapping sequence flows



- Avoid confusing twist/turns in the sequence and message flow connectors



- Avoid overlapping process steps – for extended process steps, expand the canvas and arrange the process steps neatly



- Use moduralization wherever possible – complex logic to be broken down into smaller subprocesses, named accordingly for readability. Each subprocess should contain a single function.
- Clear headers and properties after usage, in flows. Reduce the assignment of whole XML messages to a header or a property unless necessary.
- Where possible, limit the total number of steps in integration flow to 10 and use the steps local integration process to modularize complex integration flows for reducing TCO and ease of maintenance.

~~6.1.6.~~6.1.7. Concurrent Development

Where possible, in order to support concurrent development across Cloud Integration, the follow best-practices are to be followed:

- Denote work in progress artefacts with “WIP“ in the Version name. Use the Owned By field on integration flows to indicate developer ownership.
- Correctly use the Version numbers to denote releases and readiness for transport.

~~6.1.7.~~6.1.8. Script Collection

Script Collection to be used to bundle multiple scripts which can be reused across integration flows with respect to the package, to avoid the re-creation of similar scripts.

~~6.1.8.~~6.1.9. Parallel Processing – Splitter

The Splitter function within cloud integration is to be used for all batch type interfaces, where volumes are expected to reach >1000 records per run. The Splitter function splits content into packages of 1,000 entries which can then be run in parallel using the Parallel Processing function.

~~6.1.9.~~6.1.10. Integration Guidelines for Payload Logging

Payload logging is to be maintained as per Cloud Integration logging mechanisms – through the levels of Info, Debug, Trace. In general, full payload logging should not be enabled (through Monitoring or via scripting) on production environments due to data sensitivity restrictions.

The Logging of payload can be controlled Via Externalized parameter and Groovy Script within the flow itself.

~~6.1.10.~~6.1.11. External Configuration Parameters

Environment specific parameters for communication channels should always be externalized instead of using static URL(S)/Authentication details for easier and central management and maintenance.

Externalizing parameters is generally to be used where integration content is used across multiple landscape or where the endpoints of the integration flow can vary in each landscape (dynamic endpoints).

Always externalize the parameters which will change based on the environment. Example: Host Names, URL in Development, Testing or Production etc.

~~6.1.11.~~6.1.12. Restarting Messages

In Cloud Integration, when message processing fails, there is no means to retry the message processing automatically. It must be resent from the source system. Poll-type interfaces will be designed in a way to ensure missed messages are captured on subsequent runs. Push-type interfaces will be designed in a way to ensure that missing acknowledgements to source systems will trigger a re-run.

~~6.1.12.~~6.1.13. JMS Approach

In many cases integration scenarios must be decoupled asynchronously between sender and receiver message processing to ensure that a retry is done from the integration system rather than the sender system.

This would be achieved by using JMS queues to temporarily store the messages in the cloud integration system if the receiver system cannot be reached and retry them from there. Additionally, refer product documentation here.

~~6.1.13.~~6.1.14. Data Store Approach

The messages are persisted in data store for many days (as configured in the process step – default being 90 days); or a variable which stays in the database for four hundred days after the last access. In this approach, messages are persisted for pre-defined period in Cloud Integration Data Store and automatically restarted after failures. You can also configure number of retries in a global variable. Additionally, refer product documentation here.

~~6.1.14.~~6.1.15. Centralized Reusable Value Mapping API

Value mapping needs to be maintained and created in a central repository i.e., in one value mapping package for promoting reusability and should be accessed using a script for the source values in the integration flow to map it to the target value. Value maps can be accessed programmatically from a script with the help of the getMappedValue API of the ValueMappingApi class. Make use of variables where generic configuration may occur.

~~6.1.15.~~6.1.16. ~~iFlow~~ Flow Reusability

Within SAP Cloud Integration, reuse is achieved through a structured approach to artefact design that separates concerns across logical layers, governs how integration flows interact with subordinate processes and shared artefacts, and organises the overall landscape into maintainable, domain-aligned packages.

- Complex or reusable processing logic within an integration flow is to be contained within a **Local Integration Process**. This can be called as modular sub-process flows from the main integration process.
    - This keeps the primary flow concise and readable, and promotes reuse of common processing steps — such as error handling routines, retry logic, or payload enrichment — across multiple steps within a scenario without duplicating the underlying logic.
- Reusable artifacts not scenario-specific (such as message mapping, scripts, etc.), are maintained for use across integration flows rather than embedded within individual flows.
    - This ensures that a change to a shared mapping or script is made once and reflected consistently wherever that artefact is referenced. Integration flows reference these shared artefacts by dependency, and any update to a shared artefact is subject to the same versioning and promotion governance as primary integration flows to prevent unintended impacts across consuming scenarios.

7. Integration Transport and Deployment

Integration content within Cloud Integration will exist in distinct tenants, according to the environment stage (Development, Test and Production). As content is built, tested and deployed across subaccount tenants, Project Apollo will follow the below approaches for integration and deployment mechanisms in BTP.

7.1. Deployment Approach for Integrations

Integration artefacts follow a linear promotion path across three dedicated SAP Cloud Integration tenants: Development, Quality, and Production — each isolated with its own credentials, security artefacts, and endpoint configurations managed via externalised parameters. This ensures no environment-specific configuration is hardcoded within integration flows, and that the same artefact is promoted without modification.

Where possible and appropriate, a blue/green release strategy is adopted for production deployments, whereby the existing integration flow remains live while the updated version is deployed in parallel and validated under real message volumes before traffic is switched. This eliminates downtime and provides a clean fallback without requiring a re-deployment under pressure. Where parallel operation is not practical, a canary release approach is applied, routing a controlled subset of traffic to the updated flow prior to full cutover. All production deployments are scheduled within agreed change windows and coordinated with Service Stream and S/4HANA release cycles.

Rollback capability is treated as a first-class concern. The Cloud Integration tenant retains prior artefact versions in each tenant, enabling rapid redeployment. Any rollback is treated as a formal change event with a root cause analysis to follow. For stateful scenarios involving Data Store or JMS queue persistence, rollback procedures account for in-flight messages to prevent data loss or duplication.

Automated regression testing is embedded into the pipeline at two levels: artefact-level tests validate individual iFlow logic against mock endpoints, while end-to-end tests exercise critical scenarios across the full landscape upon promotion to the Quality tenant.

7.2. SAP Cloud Transport Management Service

The SAP Cloud Transport Management (CTMS) service provides the SAP-recommended enterprise-grade transport mechanism for CPI artefacts. This service will be configured to align with the Project Apollo subaccounts and transport paths.

Within CTMS, Transport requests serve as the fundamental unit of change management. Each request will contain:

Transport Request Metadata

- Unique transport request identifier (system-generated)
- Business description and change justification
- Requestor identity and timestamp
- Associated change ticket or user story reference
- Target deployment schedule
- Priority and urgency classification

Integration Artefact Inventory:

- Integration package names and semantic versions
- Package dependencies and prerequisite versions
- Source node from which packages were exported

- Package checksums for integrity verification

Approval Workflow State

- Draft: Request created but not submitted for approval
- Awaiting Approval: Submitted and pending reviewer action
- Approved: Authorised for transport execution
- In Transit: Currently executing transport to target node
- Completed: Successfully transported and deployed
- Rejected: Declined by approver with documented rationale
- Failed: Transport execution encountered errors requiring remediation

Audit Trail Entries documenting every state transition, approval decision, transport execution event, and deployment outcome. Audit entries are immutable and timestamped, providing complete forensic reconstruction capability

Project Apollo will use SAP Cloud Transport Management Service (CTMS) as the standard transport mechanism for BTP developments, including SAP Cloud Integration artefacts. CTMS will manage the movement of integration content across subaccount environments (Development, Test and Production) using defined transport routes and controlled deployment steps.

SAP Cloud ALM (CALM) will be used to provide visibility of integration changes across environments and support coordination of deployment activities.

In this model:
• CTMS manages transport execution and artefact movement across environments
• CALM provides visibility and coordination of deployments
• SAP Cloud Integration provides runtime monitoring and operational visibility for integration flows

This approach provides a consistent and controlled way to manage integration deployments, with clear visibility of changes across environments and reliable handling of integration artefacts.

7.3. Version Control Approach

SAP Cloud Integration provides native, built-in versioning capability that serves as the primary mechanism for version control across all integration artefacts for Project Apollo. Because all integration development is performed directly on the Cloud Intehgration tenant — rather than locally on developer workstations — there is no requirement to integrate with an external source control system such as Git. This is consistent with the standard operating model for SAP Cloud Integration, where the platform itself is the authoritative system of record for artefact development and versioning.

Within Cloud Integration, every integration flow, message mapping, and reusable artefact can be explicitly versioned at the point of save or deployment. Each version is accompanied by a mandatory version comment, which the programme uses to capture a concise description of the change, the associated change request or ticket reference, and the author. This provides a clear, auditable history of every modification made to an artefact over time. Prior versions are retained on the tenant and can be restored directly from the version history, supporting both rollback and traceability requirements without reliance on external tooling.

Project Apollo enforces a versioning convention aligned to semantic versioning principles — major versions for breaking changes to interfaces or message structures, minor versions for functional enhancements, and patch versions for defect fixes or configuration adjustments.

Commented [RS6]: @William Soehendra Updated now

8. Interface Security Requirements

8.1. SAP Cloud Integration Security

8.1.1. Authentication Mechanisms

SAP Cloud Integration offers a range of authentication methods when accessing integration flow endpoints (adapter sender channels), connecting to external systems via adapter receiver channels, as well as when consuming public OData APIs. Depending on the adapter and channel type, the following authentication options are supported:

- Public Key Authentication (e.g., SFTP)
- Client Certificate Authentication
- OAuth Authentication (token based, e.g., OData, AMQP)
- SASL (e.g., AMQP)
- Principal Propagation
- Basic Authentication (to be used only in non-productive scenarios, and only with risk mitigation controls in place for productive scenarios)*
- None (e.g., OData, AMQP, HTTP, SOAP, …)

As part of Project Apollo, interfaces wil **use the OAuth2.0 as the standard for authentication** where applicable. Where no strong authentication is possible, other means for securing communication will be considered, and approved by Service Stream, on an integration-by-integration bases – e.g., via signatures and encryption on message level

8.1.2. Transport Layer Security

To ensure secure communication with remote systems, recommendation is to use encrypted protocols, if supported by the system. **Transport Layer Security** (TLS) protects data in point-to-point communications in the following ways:

- Encryption of the transmitted data prevents eavesdropping.
- Secured protocols also include verification of the hostname that prevents man-in-the-middle* attacks.

SFTP Protocol: Prefer SFTP over FTP

SSH is used to securely transfer files in an open network. Supported authentication methods:

- Username/password authentication, and;
- Public key authentication (where the SFTP server authenticates the calling component based on a public key)

For SFTP integration, both Username/Password and Public Key authentication can be used in conjunction from Cloud Integration. This is the preferred approach where possible.

HTTP(S) Protocol: Prefer HTTP(S) outbound over HTTP. Inbound connections are always protected via HTTP(S)

- Protect communication using Transport Layer Security (TLS) where Default length of asymmetric keys provided by SAP is 2048 bits.
- Various authentication options (basic authentication using user credentials, client certificates, or OAuth) are supported

- **SMTPS/IMAPS/POP3S**: These protocols are supported for the exchange of e-mails (in combination with the Mail adapter). Transport encryption is supported via the STARTTLS extended operation.

8.1.3. Message Level Security

Message Level Security allows us to verify the authorship and integrity of the received message payload. Both, encryption and signing in SAP Cloud Integration are based on industry-standard encryption algorithms, such as AES, RSA, and SHA, to protect data at rest and in transit, algorithms that require a key pair consisting of a private and public key.

- For encryption, the sending party uses the public key of the receiver and only the receiver can decrypt the message with its private key.

8.1.4. Additional Security Measures

Project Apollo integration will adopt the below additional security measures.

- Avoid Data Leaks - Remove Sensitive or Secret Content Before Logging or Forwarding
- Use CSRF (Cross-Site Request Forgery) Protection - Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they are currently authenticated. Implementation Details here
- Validate input data: Validate input data to prevent injection attacks, such as SQL injection or cross-site scripting (XSS). Implement input validation mechanisms, such as parameterized queries or input sanitization
- Use the in-built Secure Credential Store, which enables users to securely store and retrieve credentials, certificates, and keys.
- Upload WSDLs as Integration Flow Resource
- Protecting Data Integrity using Message Digest
- Implement Access Control - Provide role-based access control (RBAC) to enable administrators to define user roles and assign privileges based on those roles.
- Auditing and Logging
- Perform regular security assessments
- Alignment with Service Stream ISMS Document ‚Cryptographic Encryption and Keys Management Standard v2.1 0725‘ <Link>
- Alignment with Service Stream ISMS Document ‚Application Programming Interface (API) Security Standard v1.0 0725‘ <Link>
- Alignment with Service Stream ISMS Document ‚Data Backup and Archiving Standard 2.1 0725‘ <Link>

9. Interface Monitoring

Commented [WS7]: We will add details in this section following integration workshops and monitoring requirements.

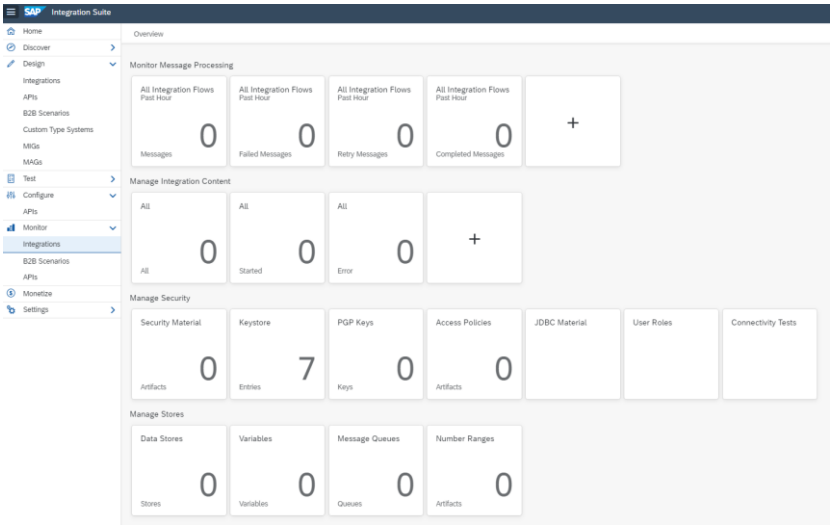
9.1. SAP Cloud Integration Message Monitoring

SAP CI includes functionality built-in to monitor and log integration runtimes, capturing key information at five levels of detail.

For integrations built and deployed in SAP CI, this message monitoring function is preferred as a first-point of call for technical monitoring and troubleshooting.

9.1.1. Message Monitoring

Message monitoring view is used to check the status of recently processed messages. In case there are lot of messages, a time and/or integration flow should be specified to monitor to narrow down the search results.



It provides counts as well as status of processed messages within a specified time window. In message monitoring, the following status of a message can be checked:

- **Error**: Error occurred during the message processing, but message retry not started automatically.
- **Failed**: Message processing failed, re-try not possible.
- **Retry**: Error occurred during message processing and retry started automatically.
- **Completed**: message processing completed, and the message delivered to the receiver successfully.
- **Cancelled**: we can cancel the messages from the message monitoring editor.
- **Processing**: message is currently being processed.
- **Escalated**: Error occurred during the message processing, but message retry not started automatically. For synchronous interfaces error messages sent to the sender.
- **System Log:** This monitoring is useful to search for the root cause of a message processing issue, the server log can be checked for more details. Each CI node has its own server log.

Since message processing only happens on IFLMAP nodes, we are only interested in the IFLMAP nodes logs. Using “System Log” view you can download the most recent part of the log.

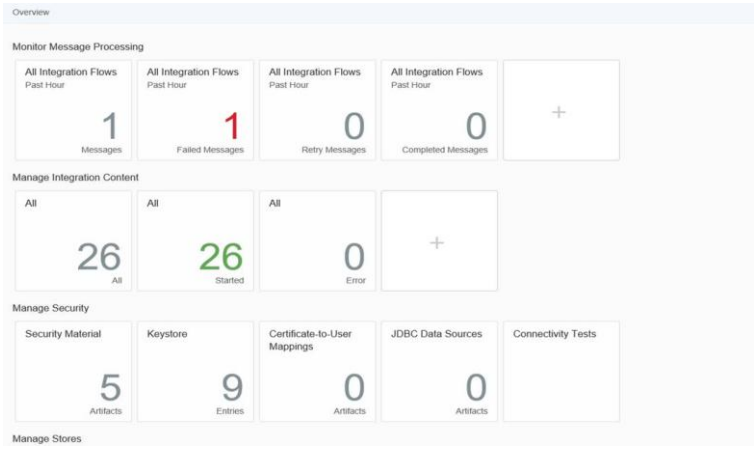
- **Message Tracing:** Useful when the payload of processed messages needs to be accessed. When activated, the message tracing gathers payload and headers on each step during Integration flow execution. You can access this information later from the message monitoring. Below table shows the various log levels:

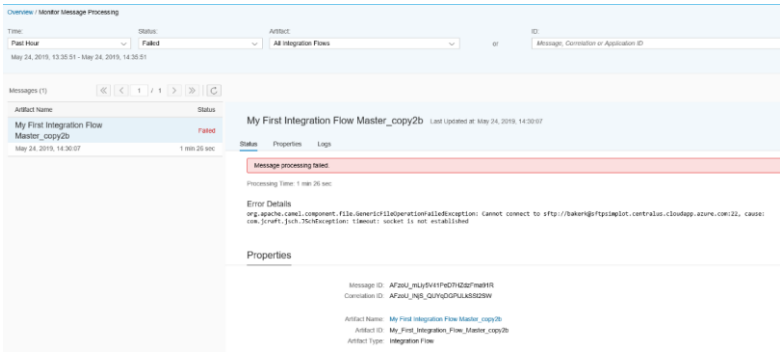
Log Level	Description
None	No data is recorded during message processing and no data is shown in data monitoring.
Info	Basic information is recorded during message processing. The header is always displayed. If there are failed messages, the message processing log retains and displays additional detailed information about the last 50 message steps.
Error	The message processing log records basic information for failed message executions only. The message processing log retains and displays detailed information about the last 50 message steps. Use this log level, if you've no interest in recording completed messages. No message processing log attachments are recorded.
Debug	As the log level Debug is a high resource consuming feature, we recommend setting log level Debug only in a development or test environment! The message processing log records detailed information for all steps during message processing. While all steps (with headers and additional information) are accessible in the graphical UI, only the last 100 are shown in the text view of the message processing log. The log level debug expires after a certain time. After expiry, the log level switches back to the log level set before. **Note:** In the Cloud Foundry environment, the expiry time is fixed and set to 24 hours.
Trace	Trace is a high resource consuming feature; we recommend setting log level Trace only in a development or test environment! SAP can disable this function for a specific tenant or adjust expiry and retention time. The message processing log records detailed information for all steps during message processing and additionally tracks the message content. The trace function expires after a certain time (default value: 10 minutes). After expiry, the log level switches back to the log level set before. The recorded message content is also only retained for a certain time (default value: 1 hour)

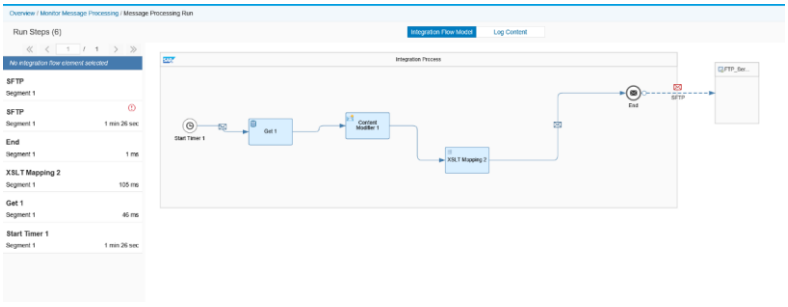
- **MPL Attachments:** CI keeps a message processing log for every processed message. The log contains the basic information: status, error message, and start/end timestamps for Integration flow components. Additional entries into the log can be inserted using custom script (Groovy or JavaScript). There are two possibilities:
    - Properties - Simple name/value pairs written into the log attachments
    - Attachments - Complete documents which are ‘attached’ to the message processing log. You can store the entire payload or other document as attachment

  In the below screenshot, the message monitoring tiles reveal messages with a status of either “Failed / Error / Completed” that have been processed for all the integration scenarios within

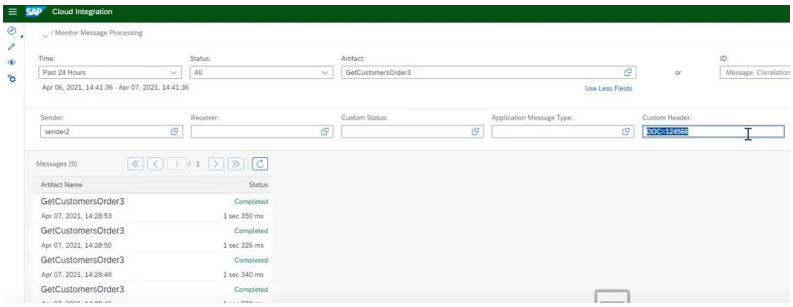
the past 24 hrs. A Number representing the no. of messages in that status. Each tile shows the filter settings in the header and the number of messages and artifacts that corresponding to the filter settings.





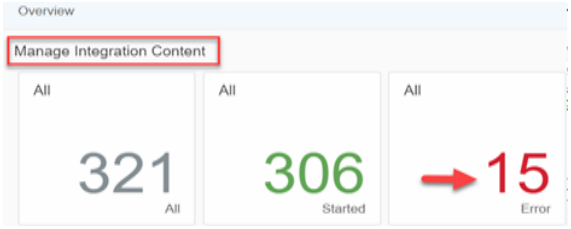


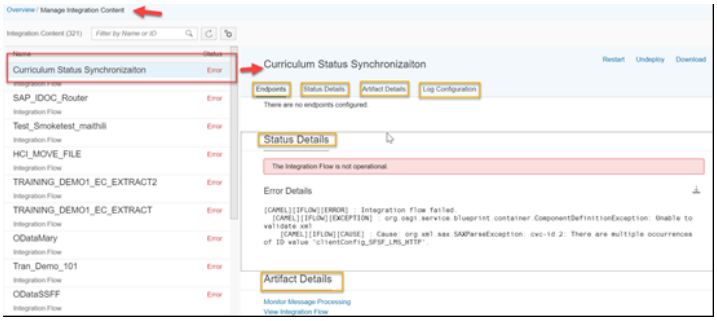
- **Custom Header Search**: Custom headers are additional properties that can be added to Integration flow runtime to provide additional information or customization. When performing a custom header search, one examines runtime message to locate the desired custom headers. This search enables users to locate and manipulate custom headers for various purposes, such as authentication, authorization, data transformation, or integration with external systems.



9.1.2. Managing Integration Content

The Integration Manage view on Cloud Integration provides overview of the integration artifacts such as integration flows and security related artifacts deployed on the tenant with all statuses. We can also display the status of individual artifacts. There is also provision to **restart** an Integration flow, **undeploy** an already started integration flow or even **download** artifacts of an integration flow

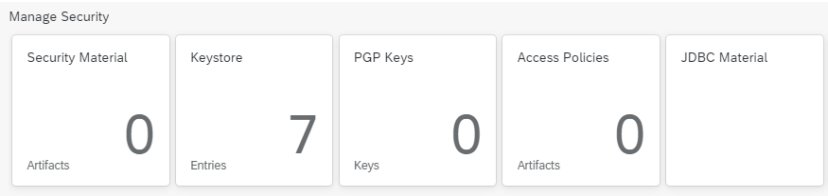




9.1.3. Managing Security Material

SAP CI provides a centralized location for managing certificates, keystores, trust stores, and other security materials used for authentication, encryption, and secure communication between CPI and

external systems or partners. With the "Manage Security Material" functionality, users can easily upload, view, update, and delete security materials, ensuring the secure and reliable exchange of data between CPI and various endpoints. It simplifies the management of security components and helps ensure the confidentiality, integrity, and authenticity of data during integration processes.



9.1.4. Managing Data Stores

SAP CI provides the capability of managing various data-related components, including data stores, variables, message queues, and number ranges

Data Stores: Data stores are used to persist and retrieve data during integration flows. They provide a way to store and access data between different steps or executions of an integration process. Users can define and configure data stores and handle data operations such as reading, writing, and querying data.

Variables: Variables allow for temporary storage of data within integration flows. They are used to hold and manipulate data during the execution of an integration process. Users can define and manage variables, assign values, perform transformations, and utilize them for mapping, conditional logic, or calculations within integration flows.

Message Queues: Message queues facilitate asynchronous message processing and decouple systems in distributed environments. They enable reliable messaging and ensure messages are processed in a guaranteed order. Users can create and configure message queues, define their properties, set up subscribers and publishers, and handle message retrieval and processing.

Number Ranges: Number ranges are used to generate unique identifiers or sequential numbers for various purposes within integration flows. They are commonly utilized for generating order numbers, invoice numbers, or unique transaction IDs. Users can define and manage number ranges, set initial values, specify increment steps, and retrieve the next available number when required.

10. Alerting and Error Handling

Integration runtime alerting and error handling is to be processed within Cloud Integration internally, rather than centrally from Service Stream's central SIEM platform (Splunk) – this is documented in JIRA Decision ETF-157.

10.1. SAP Cloud Integration

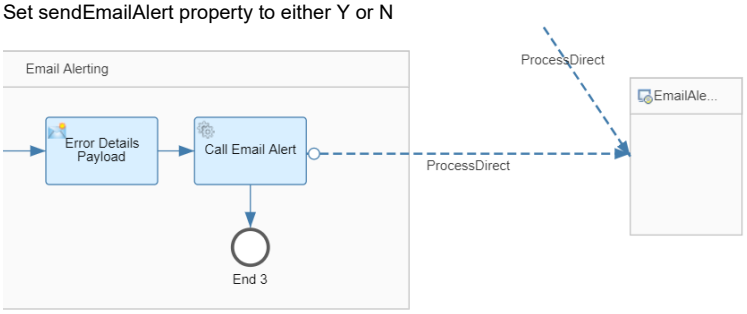
Exception Handling is a design time process and should be adhered to on how to handle exceptions in the integration flow. By making use of **exception sub-process** in integration flow we can capture the exception thrown in the integration flow and effectively handle by assigning next steps of action.

- Every integration flow should have an exception sub-process instantiated with steps to handle exceptions clearly defined. This will ensure consistent/graceful handling of exceptions that occur.
- If making use of Local Integration Processes, we need to define the exception sub-process separately for each Local Integration Process.
- Depending on whether the Exception Subprocess in the parent integration flow ends with an End Message or an Error End event, in the exception case the integration process gets a different status during monitoring:
    - **End Message event:** If an exception occurs, parent integration flow ends with status Completed, which is the preferred method for synchronous integration after the error was reported back to the sender.
    - **Error End Event:** If an exception occurs, parent integration flow ends with status Failed, which is the preferred method for asynchronous integration.

Re-usable alerting process (using the Process Direct Adapter): this subprocess is to be used across integration flows as a standard alerting mechanism.

Main integration flow:

1. Set sendEmailAlert property to either Y or N



2. Payload example via content modifier (add as much details as possible into emailBody)

```
<ErrorEvent>
  <MPL_ID>${property.SAP_MessageProcessingLogID}</MPL_ID>
  <emailSubject>iFlow Name</emailSubject>
  <emailBody>
<![CDATA[Error Time: ${date:now:dd-MM-yyyy HH:mm z}
Error Message: ${exception.message} ]]>
</emailBody>
  <sendEmail>${property.sendEmailAlert}</sendEmail>
</ErrorEvent>
```

10.2. Business Based Exception Handling

Integration exceptions and alerts should be designed to correlate clearly to real-world business impacts, so that technical failures can be understood and prioritised in business terms such as failed or delayed Work Orders, Purchase Orders, invoices, goods movements or other critical transactions.

At a strategy level, this means exception handling should support traceability from the integration event through to the affected business object, enabling operations and support teams to identify what has failed, who is impacted and what action is required. Where appropriate, this visibility should be surfaced through business-facing tickets, workflow tasks or notifications, and/or linked back to enterprise IT service management and finance ticketing platforms to support coordinated triage, ownership and resolution across business and technical teams. The intent is to provide end-to-end transparency and accountability for integration-related issues while aligning severity, routing and escalation to business impact.

Detailed requirements for specific business-related exception handling, alerting, correlation rules, ownership and resolution processes are to be defined in the Functional Specification for each individual integration.

Ongoing business escalation and exception handling, with service levels, is to be defined in the enterprise support model and incident management frameworks for Project Apollo.

11. Integration RACI

For Project Apollo, the design, build, testing, deployment and maintenance of integrations will involve systems in the SAP enviornment, systems in the existing Service Stream landscape, and systems external.

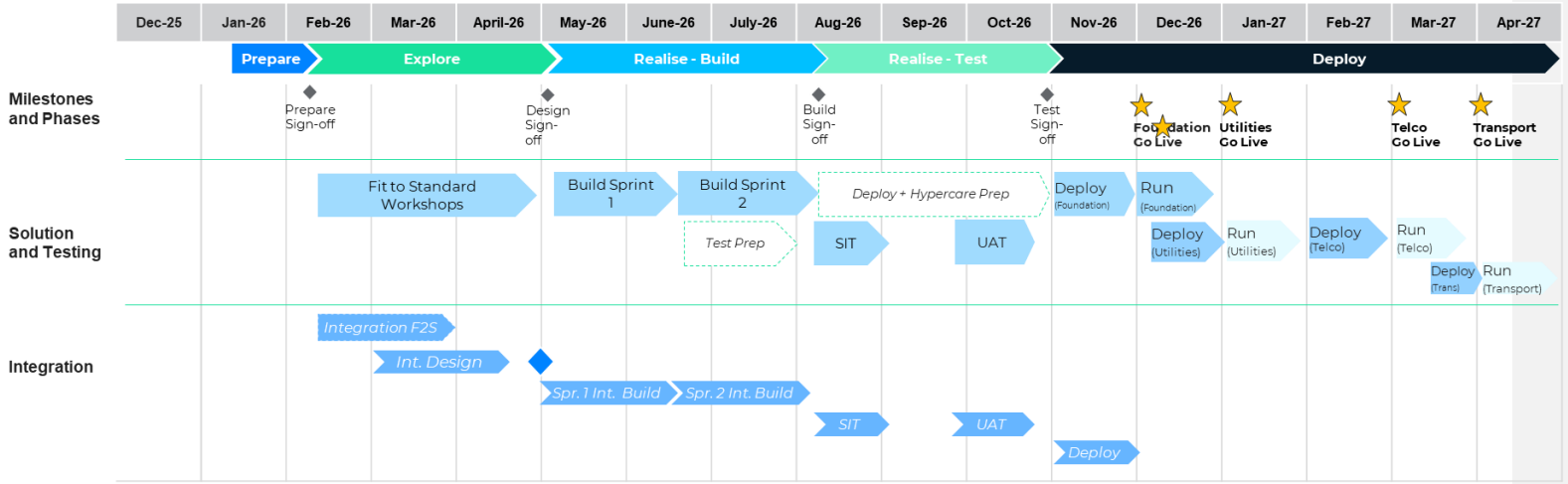
Therefore, the below RACI matrix outlines the responsibilities and ownership of tasks.

Activities	Service Stream		Deloitte
Advise on Service Stream Integration / Enterprise Architecture standards and guidelines	RA		CI
Legacy system impact assessment	RA		CI
Legacy system design / build activities (if required)	RA		CI
Design, Build, Test and Deploy SAP Integration Layer build from SAP to Middleware, for above scoped integrations	CI		RA
Design, Build, Test and Deploy SAP Integration Layer build from Middleware to Third Party / BAU System, for above scoped integrations	RA		CI
Manual activities associated with interim states related to the upload / download / transfer of data between S/4HANA and legacy systems	RA		CI
Maintenance of interim state mapping tables from end of Foundational Go Live hypercare onwards	RA		CI
Enablement of APIs and File-generation (and necessary modifications, if required) on source systems	RA		CI

No interim-state interface design or development is in the scope of this project. Any manual processes required to support interim or transitional states including their design, execution, and operation will be the responsibility of Service Stream. Deloitte will provide the following support for the interim state:

Phase	Activities
Explore	Document interim key process flows (to level 2), control points and risks and controls. In Detailed Design Documents. Signavio Business Processes for interim states will not be delivered unless minor adjustments are required and feasible to be delivered.
Realise Build	Build of up to 3 mapping tables to support interim state Master Data alignment.
Realise Test	Control points for SAP related interim states defined in Design Documents will be included in test scripts.
Run	General support for manual uploads/downloads in S/4HANA related to the interim state will be provided during hypercare but performed by Service Stream.

12. Integration Timelines



13. Open Items

JIRA ID	Title	Due By	Assignee
ETF-230	Investigate SiteTracker ability to notify middleware based on Project/Job events	27/03/2026	Rajesh Sha
ETF-129	Confirm S/4HANA Environment Management Strategy	06/03/2026	Rajesh Sha
ETF-40	Confirm if Boomi interface will be enhanced to map legacy GL accounts to SAP GL Accounts for Blackline integration	02/04/2026	Stuart Downes
ETF-237	Confirm requirement for SAP Digital Payments or Custom Built Solution for processing card related payments in the Digital Store	03/04/2026	Rajesh Sha
ETF-442	Payment Service Provider for SSM Digital Stores	-	Rajesh Sha

**This open items list will be updated as JIRA items are closed or updated.*